

Plano de Reconstrução — Cur10usX

Versão: 2.0 (Rebuild)

Equipa: 5 Pessoas

Stack Alvo: NestJS + Next.js + Prisma + PostgreSQL + WebSocket + Redis + Docker

Data: Maio 2026

Índice

1. [Filosofia da Reconstrução](#)
 2. [Stack e Arquitetura Alvo](#)
 3. [Estrutura de Equipa \(5 pessoas\)](#)
 4. [Organização do Projeto por Módulos](#)
 5. [Dependências entre Módulos](#)
 6. [Estratégia de Monorepo](#)
 7. [Estratégia de Git](#)
 8. [Organização de Sprints](#)
 9. [Plano de Desenvolvimento Paralelo](#)
 10. [Pipeline de Desenvolvimento e Deploy](#)
 11. [Estratégia de QA e Testes](#)
 12. [Estrutura de Documentação](#)
 13. [Riscos Técnicos e Mitigação](#)
 14. [Checklist de Execução](#)
-

1. Filosofia da Reconstrução

Porquê reconstruir?

O código atual (~45k linhas, 159 API routes) foi construído por um único developer. É funcional, mas tem problemas estruturais típicos de projetos individuais:

- **Acoplamento:** Lógica de negócio misturada com handlers HTTP
- **Inconsistência:** Padrões diferentes entre endpoints semelhantes
- **Testabilidade reduzida:** Lógica embutida em route handlers
- **Dívida técnica:** Acumulada por iterações rápidas sem refactoring

Princípios da Reconstrução

1. **Código existente como referência, não como base** — Vamos reescrever, não refatorar
2. **Separation of Concerns** — Camadas bem definidas (Controller → Service → Repository)
3. **Testes primeiro (onde possível)** — TDD para módulos críticos (auth, evaluation)
4. **Paridade funcional** — A versão 2.0 deve ter EXATAMENTE as mesmas features da v1
5. **Melhoria contínua** — Só introduzimos melhorias estruturais, não mudanças de scope
6. **Documentação como pré-requisito** — Cada módulo documentado antes de codificado

O que NÃO muda

- Database schema (Prisma) — O schema atual é bem modelado
- Experiência do usuário — UI/UX é preservada
- Funcionalidades — Mesmos 28 pontos de módulos
- Dados — Migração de dados existentes para o novo schema

O que MUDA

Backend	API Routes Next.js	NestJS (modular)
Frontend	Next.js App Router	Next.js App Router (melhorado)
Separação	Monolito acoplado	Monorepo com packages separados
API Pattern	Handler functions	Controller + Service + Repository
Validação	Zod inline	DTOs + Validation Pipe
WebSocket	ws server standalone	NestJS Gateway + Redis adapter
Testes	3 arquivos	Cobertura >60%
Documentação	Dispersa	ADRs + API Docs + Runbooks
Error Handling	Inline try/catch	Global filters + interceptors

2. Stack e Arquitetura Alvo

Decisão Arquitetural Crítica

Porquê NestJS em vez de manter API Routes?

Modularidade	N/A (ficheiros soltos)	Modules + Decorators
Testabilidade	Difícil (HTTP 耦合)	Fácil (DI + TestBed)
WebSocket	Servidor externo	Gateway nativo
Validação	Manual	Pipes automáticos
Documentação API	Manual	Swagger/OpenAPI automático

Middleware Edge + inline Guards + Interceptors + Pipes

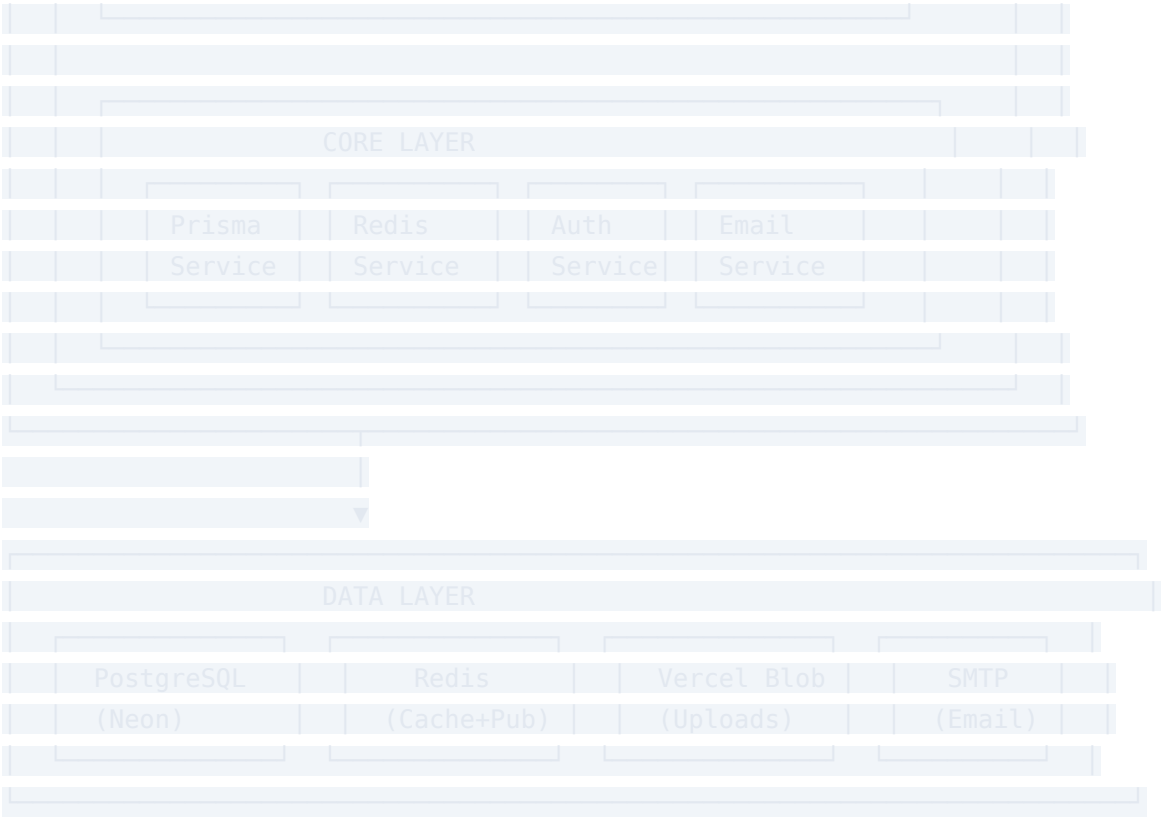
Escalabilidade Monolito Microsserviços futuros

Decisão: NestJS para backend puro, Next.js para frontend puro.

Comunicação via REST + WebSocket entre eles.

Arquitetura Final





Stack Completa

Frontend Framework	Next.js	16.1.6	App Router, Server Components, SSG
UI	React	19.2.3	Server/Client Components
Styling	Tailwind CSS	4	Build-time, zero runtime
Componentes	shadcn/ui + Radix	—	Acessível, customizável, 40+ componentes
Estado Global	Redux Toolkit	2.x	Previsível, middleware, devtools
Backend Framework	NestJS	11.x	Modular, DI, Decorators, OpenAPI
ORM	Prisma	6.x	Type-safe, schema-first, migrations

Database	PostgreSQL (Neon)	16	Serverless, branching, escalável
Auth	NestJS Passport + JWT	—	Estratégias modulares, guard-based
Validação	class-validator + Zod	—	Pipes automáticos + schemas
WebSocket	NestJS Gateway + Redis	—	Gateway nativo, adapter Redis
Cache/Queue	Redis (ioredis)	7	Cache, pub/sub, rate limiting
API Docs	Swagger/OpenAPI	—	Geração automática via decorators
Email	Resend	6.x	API moderna, React Email templates
Storage	Vercel Blob	—	CDN, edge uploads
Charts	Recharts	3.x	React-native, responsivo
PDF	jsPDF + jspdf-autotable	—	Certificados, relatórios
Container	Docker + Compose	—	Consistência, dev/prod parity
CI/CD	GitHub Actions	—	Lint, test, build, deploy

3. Estrutura de Equipe (5 pessoas)

Roles (segundo subject)

M1 — Você	Tech Lead / Architect	DevOps
M2	Product Owner	Frontend Developer

M3	Project Manager / Scrum Master	Backend Developer
M4	Backend Developer	—
M5	Frontend Developer	QA

Matriz de Responsabilidades (RACI)

Arquitetura técnica	R	C	C	I	I
Definição de features	C	R	A	C	C
Backend Core (Auth, Prisma)	R	I	C	A	I
Frontend Core (Layout, Design System)	I	A	C	I	R
Módulo Académico	I	C	R	A	C
Chat / WebSocket	R	C	A	A	I
Admin / Permissions	C	A	R	A	C
Analytics Dashboard	I	A	C	R	A
i18n / Acessibilidade	I	R	C	C	A
DevOps / Docker / CI	R	C	A	C	C
Testes E2E	I	C	A	C	R
Documentação	A	C	R	C	A

Legenda: R = Responsável, A = Aprova, C = Consultado, I = Informado

Responsabilidades Detalhadas

M1 — Tech Lead & DevOps

- Definir arquitetura e padrões técnicos
- Configurar monorepo, Docker, CI/CD
- Implementar módulo core (Auth, Prisma, Redis, WebSocket gateway)
- Code review de TODOS os PRs
- Resolver bloqueios técnicos da equipa
- Garantir performance e segurança
- Onboarding técnico dos membros

M2 — Product Owner & Frontend

- Manter product backlog (GitHub Issues)
- Priorizar features com a equipa
- Validar entregas (review de funcionalidades)
- Implementar landing page, auth pages, layout global
- Sistema de design (40 componentes UI)
- i18n (4 idiomas)
- Comunicar com stakeholders (avaliadores)

M3 — Scrum Master & Backend

- Facilitar daily standups, planning, retro
- Remover bloqueios da equipa
- Tracking de progresso (sprints)
- Implementar módulo de gestão académica (turmas, disciplinas, cursos, matrículas, avaliações)
- Motor de avaliação (grading config, trimestres, médias)
- Módulo de presenças, exames, trabalhos

M4 — Backend Developer

- Implementar CRUDs de todas as entidades (30+ models)
- Módulo de chat, mensagens, anúncios
- Módulo de amizades e notificações
- API pública (5+ endpoints com rate limiting + API key)
- Import/export de dados (CSV, XLSX)
- Suporte a módulo de analytics

M5 — Frontend Developer & QA

- Implementar todas as páginas de listagem (21 páginas)
 - Formulários CRUD (20 forms)
 - Dashboard analítico (Recharts)
 - Admin panel (11 sub-seções)
 - Testes E2E (Playwright)
 - Testes de regressão manuais
 - Report e tracking de bugs
-

4. Organização do Projeto por Módulos

Monorepo Structure (Turborepo)

```
curl0usx/  
├── .github/  
│   └── workflows/  
│       ├── ci.yaml          # CI: lint, test, build  
│       └── cd.yaml          # CD: deploy  
├── docker/  
│   ├── Dockerfile.frontend  
│   ├── Dockerfile.backend  
│   └── docker-compose.yaml  
├──  
├── packages/  
│   └── frontend/            # Next.js 16 App Router
```



```

├── package.json
├── tsconfig.json
├──
├── docs/ # Documentation
├──   ├── ARCHITECTURE.md
├──   ├── API.md # OpenAPI/Swagger export
├──   ├── DATABASE.md
├──   ├── DEPLOYMENT.md
├──   ├── SPRINT_X.md # Sprint plans
├──   └── ADR/ # Architecture Decision Records
├──     ├── 001-use-nestjs.md
├──     ├── 002-monorepo-structure.md
├──     └── ...
├──
├── .env.example
├── .gitignore
├── .eslintrc.js
├── .prettierrc
├── turbo.json # Turborepo config
├── Makefile
└── README.md

```

Módulos do Negócio vs Módulos 42

Cada módulo do código mapeia para um ou mais módulos de avaliação:

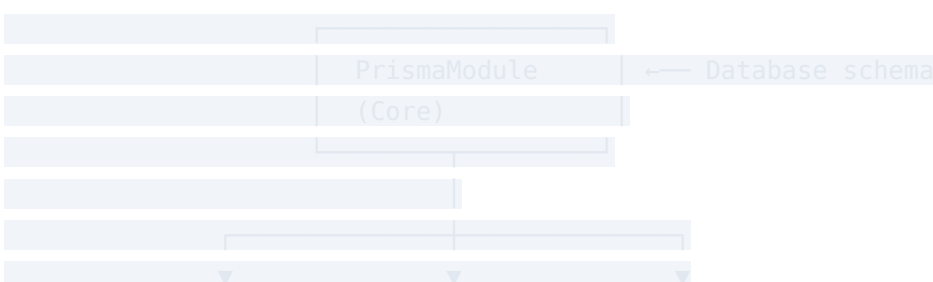
auth/	User Management (Standard)	2
auth/ (OAuth)	User Management (OAuth 2.0)	1
auth/ (2FA)	User Management (2FA)	1
users/ + friends/	Web (User Interaction)	2
school/	User Management (Organization)	2
admin/	User Management (Advanced Permissions)	2
chat/ + messaging/	Web (Real-time) + Advanced Chat	2 + 1
notifications/	Web (Notification System)	1

academic/	— (core business logic)	—
evaluation/	— (core business logic)	—
analytics/	Data (Analytics Dashboard)	2
files/	Web (File Upload)	1
import-export/	Data (Export/Import)	1
gdpr/	Data (GDPR)	1
frontend (design system)	Web (Custom Design System)	1
frontend (i18n)	Accessibility (Multiple Languages)	1
prisma (ORM)	Web (ORM)	1
health/	DevOps (Health Check)	1
API routes (5+ endpoints)	Web (Public API)	2
—	Activity Analytics	1
—	Advanced Search	1

Total: 14+ pontos (major 8 + minor 12 = 28 pontos atuais)

5. Dependências entre Módulos

Grafo de Dependências





Ordem de Implementação

Fase 0: Setup

- Monorepo (Turborepo)
- Docker + Docker Compose
- CI/CD (GitHub Actions)
- ESLint + Prettier + Husky

Fase 1: Fundação (Semana 1-2)

- Prisma Schema + Migrations ← SEMPRE primeiro
- PrismaModule + PrismaService
- RedisModule + RedisService
- AuthModule (JWT, signup, signin)
- Common Layer (guards, pipes, filters, interceptors)
- Frontend: Design System + Auth Pages

Fase 2: Core Business (Semana 3-5)

- UsersModule (CRUD, profiles)
- SchoolModule (multi-tenant)
- AcademicModule (classes, subjects, courses, enrollments)
- FriendsModule + NotificationModule
- Frontend: Dashboard layout + CRUD pages

Fase 3: Features Complexas (Semana 6-8)

- EvaluationModule (grading engine)
- ChatModule + WebSocket Gateway

- └─ MessagingModule + AnnouncementsModule
- └─ FilesModule (upload)
- └─ Frontend: forms + detail pages

Fase 4: Advanced (Semana 9-10)

- └─ AdminModule + Permissions
- └─ AnalyticsModule
- └─ ImportExportModule
- └─ GDPRModule
- └─ Frontend: Admin panel + Analytics + i18n

Fase 5: Qualidade (Semana 11-12)

- └─ Testes: Unitários + Integração + E2E
- └─ Documentação
- └─ Performance optimization
- └─ Swagger/OpenAPI docs
- └─ Deploy + Rollout

6. Estratégia de Monorepo

Turborepo

Usar **Turborepo** para gerenciar o monorepo:

```
// turbo.json
{
  "$schema": "https://turbo.build/schema.json",
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": [".next/**", "dist/**"]
    },
    "test": {
      "dependsOn": ["^build"],
      "outputs": []
    },
    "lint": {
      "outputs": []
    },
    "dev": {
      "cache": false,
```


Package linking

- `packages/shared` → tipos, constantes, DTOs (publicado como `@cur10usx/shared`)
- `packages/backend` → depende de `@cur10usx/shared`
- `packages/frontend` → depende de `@cur10usx/shared`

Porquê monorepo?

1. **Tipos partilhados** — Mesmas interfaces entre frontend e backend
2. **Commits atômicos** — Mudanças cross-package num commit
3. **CI unificada** — Um pipeline para tudo
4. **Refactoring consistente** — Renomear tipos propaga-se automaticamente
5. **Onboarding simples** — `npm install` na raiz instala tudo

7. Estratégia de Git

Fluxo de Branches (Git Flow Simplificado)

Regras

main	Tech Lead	 Protegida	Código em produção
staging	Tech Lead + PM	 Protegida	QA e testes
develop	Qualquer dev após PR aprovado	 Protegida	Integração diária
feature/*	Criada por qualquer dev	—	Desenvolvimento
fix/*	Criada por qualquer dev	—	Bug fixes
release/*	Tech Lead	 Protegida	Preparação de release

Commits

Formato: Conventional Commits

```
<type>(<scope>): <description>
```

```
[optional body]
```

```
[optional footer]
```

Types: feat, fix, refactor, test, docs, style, chore, perf

Exemplos:

```
feat(auth): implement JWT signup and signin
```

```
fix(evaluation): correct grading calculation for weighted averages
```

```
refactor(chat): extract message broadcasting to service
```

```
test(users): add unit tests for user CRUD service
```

```
docs(api): add OpenAPI documentation for auth endpoints
```

Regras de commit:

- Commits pequenos e atômicos (1 feature = vários commits)

- Mensagens em inglês (consistência global)
- Relacionados a uma issue do GitHub

Pull Requests

Template:

```
## Descrição
<!-- O que este PR implementa -->

## Tipo de mudança
- [ ] feat (nova funcionalidade)
- [ ] fix (correção de bug)
- [ ] refactor (refactoring)
- [ ] test (testes)
- [ ] docs (documentação)
- [ ] chore (build, CI, config)

## Checklist
- [ ] Código segue os padrões do projeto
- [ ] Testes unitários adicionados/atualizados
- [ ] Testes E2E passam
- [ ] Documentação atualizada
- [ ] Próprio código revisado

## Screenshots (se aplicável)

## Notas adicionais
```

Regras de PR:

- Mínimo 1 approval de code review
- Tech Lead aprova PRs de backend core (auth, prisma, websocket)
- Frontend Lead aprova PRs de frontend
- TODOS os PRs devem passar CI (lint + test + build)
- Branches de feature duram no máximo 3 dias (evitar drift)
- PRs com mais de 400 linhas são recusados (forçar commits pequenos)

Code Review Guidelines

O que verificar:

1. Lógica de negócio está correta?
2. Tratamento de erros adequado?
3. Segurança (autenticação, autorização, validação)?
4. Performance (N+1 queries, índices)?
5. Código segue os padrões do projeto?
6. Testes cobrem os casos de uso?
7. Documentação foi atualizada?

Como dar review:

- Comentários construtivos e específicos
 - Sugerir em vez de criticar
 - Aprovar quando estiver bom, não quando estiver perfeito
 - Usar "nit:" para sugestões não-críticas
-

8. Organização de Sprints

Estrutura

- Duração: **2 semanas** cada sprint
- Cerimónias:
 - **Planning** (Segunda, 1h) — início do sprint
 - **Daily** (15min) — todos os dias
 - **Review** (Sexta, 30min) — demo do que foi feito
 - **Retro** (Sexta, 30min) — o que melhorar
- Ferramentas: **GitHub Projects** (kanban) + **Discord** (comunicação assíncrona)

Sprint 0: Fundação (Semanas 1-2)

Objetivo: Setup completo do ambiente de desenvolvimento

Configurar monorepo (Turborepo)	M1	—	1 dia
Configurar Docker + Compose	M1	—	1 dia
CI/CD (GitHub Actions)	M1	Docker	2 dias
ESLint + Prettier + Husky	M1	Monorepo	0.5 dia
Prisma Schema (30+ models)	M1+M 3	—	3 dias
PrismaModule + PrismaService	M1+M 3	Schema	1 dia
RedisModule + RedisService	M1	Docker	1 dia
AuthModule (JWT, signup, signin)	M1	Prisma	3 dias
Common Layer (guards, pipes, filters)	M1	NestJS setup	2 dias
Frontend: Next.js + Tailwind + shadcn	M2+M 5	Monorepo	2 dias
Frontend: Design System (40 componentes)	M2	Tailwind/ shadcn	4 dias
Frontend: Auth pages (login, register, 2FA)	M2	Design System	3 dias
Documentação: ADRs + Architecture	M1+M 3	—	2 dias
Onboarding dos membros	M1	—	1 dia

Milestone: App rodando local com Docker, `docker compose up` funcional, auth funcional.

Sprint 1: Core Business (Semanas 3-4)

Objetivo: Módulos principais do negócio

UsersModule (CRUD, profiles, roles)	M3	AuthModule	3 dias
SchoolModule (multi-tenant)	M3	UsersModule	3 dias
FriendsModule (CRUD, status)	M4	UsersModule	2 dias
NotificationModule (CRUD + WS)	M4	Redis, WebSocket	3 dias
Frontend: Dashboard layout (sidebar, navbar)	M2	Design System	2 dias
Frontend: Profile page + avatar upload	M2	UsersModule	2 dias
Frontend: Friend system UI	M5	FriendsModule	2 dias
Frontend: Notification center	M5	NotificationModule	2 dias
Testes: Auth + Users (unitários)	M1+M 3	—	2 dias

Milestone: Usuários podem registrar-se, fazer login, gerir perfil, adicionar amigos, receber notificações.

Sprint 2: Academic Core (Semanas 5-6)

Objetivo: Gestão académica completa

AcademicModule: Classes + Subjects + Courses	M3	SchoolModule	3 dias
AcademicModule: Enrollments + Lessons	M3	Classes + Subjects	3 dias
AcademicModule: Exams + Assignments	M4	Lessons	3 dias
AcademicModule: Attendance	M4	Lessons	2 dias
Frontend: CRUD pages (21 list pages)	M5	AcademicModule	5 dias
Frontend: Forms (20 forms)	M2	AcademicModule	5 dias

Testes: Academic (integração)	M3	—	2 dias
-------------------------------	----	---	--------

Milestone: Gestão académica funcional — turmas, disciplinas, matrículas, presenças, exames, trabalhos.

Sprint 3: Evaluation + Chat (Semanas 7-8)

Objetivo: Motor de avaliação + comunicação real-time

EvaluationModule (grading engine)	M3	AcademicModule	5 dias
Resultados + médias + certificados	M3	EvaluationModule	3 dias
ChatModule + WebSocket Gateway	M1	AuthModule, Redis	4 dias
MessagingModule (mensagens internas)	M1	UsersModule	2 dias
Advanced Chat (typing, read receipts)	M4	ChatModule	2 dias
AnnouncementsModule	M4	Notifications	2 dias
Frontend: Chat interface (real-time)	M5	ChatModule	3 dias
Frontend: Messaging + Announcements	M5	MessagingModule	2 dias
Testes: Evaluation + Chat	M3+M4	—	3 dias

Milestone: Chat real-time funcional, motor de avaliação a calcular médias.

Sprint 4: Admin + Analytics (Semanas 9-10)

Objetivo: Painéis administrativos e analytics

AdminModule (CRUD escolas, users, config)	M3	ALL modules	4 dias
Advanced Permissions (15 permissões)	M3	AdminModule	2 dias
AnalyticsModule (stats, charts data)	M4	AcademicModule	4 dias
FilesModule (upload, validation, preview)	M4	UsersModule	3 dias
ImportExportModule (CSV, XLSX)	M4	ALL modules	3 dias
GDPRModule (export, delete, confirmation)	M3	UsersModule	2 dias
Frontend: Admin panel (11 sub-seções)	M2	AdminModule	4 dias
Frontend: Analytics dashboard (Recharts)	M5	AnalyticsModule	3 dias
Frontend: File upload UI	M5	FilesModule	2 dias
Testes: Admin + Analytics	M4	—	2 dias

Milestone: Admin funcional, dashboard analítico, import/export funcional.

Sprint 5: Quality + Polish (Semanas 11-12)

Objetivo: Qualidade, testes, documentação, deploy

i18n: PT, EN, FR, ES (completar)	M2	Frontend	3 dias
Frontend: Páginas de privacidade + termos	M2	—	1 dia
Frontend: Responsividade + acessibilidade	M2+M5	Frontend	3 dias
API Pública (5 endpoints + rate limit + docs)	M4	ALL modules	2 dias

Swagger/OpenAPI automático	M1	NestJS	1 dia
Testes E2E (Playwright — 10 flows críticos)	M5	Frontend	4 dias
Testes unitários (cobertura >60%)	M3+M4	ALL modules	4 dias
Testes de integração (API)	M3	ALL modules	3 dias
Documentação README (completa, subject)	M3	ALL	2 dias
Performance optimization	M1	ALL	2 dias
Deploy + Rollout	M1	Docker	2 dias
Bug bash + Regressão	ALL	—	2 dias

Milestone: Sistema completo, testado, documentado, deployed.

9. Plano de Desenvolvimento Paralelo

Como evitar conflitos

1. **Monorepo com packages separados** — Frontend e backend em packages diferentes
2. **Shared package** — Tipos compartilhados, cada equipa trabalha no seu domínio
3. **Feature branches** — Cada feature na sua branch
4. **Definição clara de interfaces** — Contrato API definido ANTES da implementação

Paralelismo por Sprint

Sprint 1 (Semanas 3-4):

└─ M1: Tech Lead ───────────────────┘
WebSocket setup + Redis pub/sub
Code review de todos os PRs
Common layer (filters, interceptors)

API Contract-First

Antes de iniciar cada sprint:

1. M1 (Tech Lead) define o contrato da API (endpoints, request/response)
2. Contrato é documentado no shared package (@cur10usx/shared)
3. Backend implementa o contrato
4. Frontend consome o contrato (mock inicial se backend ainda não estiver pronto)

Exemplo de workflow:

Dia 1: M1 define contrato de ChatModule

Dia 1: M4 começa backend do ChatModule

Dia 2: M5 começa frontend do ChatModule (usando mocks)

Dia 4: M4 termina backend → M5 substitui mocks por API real

Git Workflow Diário

1. git checkout develop && git pull
2. git checkout -b feature/meu-modulo
3. (trabalho no código)
4. git add . && git commit -m "feat(modulo): descrição"
5. git push origin feature/meu-modulo
6. (abrir PR no GitHub)
7. (atribuir reviewer)
8. (CI roda automaticamente)
9. (reviewer aprova ou pede mudanças)
10. (merge para develop após aprovação)

10. Pipeline de Desenvolvimento e Deploy

CI/CD (GitHub Actions)

```
# .github/workflows/ci.yaml
name: CI
on:
  push:
    branches: [develop, staging, main]
  pull_request:
```

```

  branches: [develop, staging, main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npx turbo lint

  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_DB: curl0usx_test, POSTGRES_USER: test, POSTGRES_PASSWORD:
test }
        ports: ['5432:5432']
      redis:
        image: redis:7
        ports: ['6379:6379']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npx prisma generate
      - run: npx turbo test

  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npx turbo build
      - run: docker build -t curl0usx-backend -f docker/Dockerfile.backend .
      - run: docker build -t curl0usx-frontend -f docker/Dockerfile.frontend .

```

Pipeline de Deploy

Push para main

- GitHub Actions build + test
- Docker images build + push (ghcr.io)
- Deploy para staging (VPS/Cloud)
- Testes E2E automáticos (Playwright)
- Aprovação manual (Tech Lead + PO)
- Deploy para produção
- Smoke tests
- Rollback se falhar

Ambientes

development	localhost:3000	Desenvolvimento local	<code>docker compose up</code>
staging	staging.cur10usx.com	QA + testes	Automático (push staging)
production	cur10usx.com	Produção	Manual (após aprovação)

Docker Compose (dev)

```
version: '3.8'
services:
  postgres:
    image: postgres:16
    ports: ['5432:5432']
    volumes: ['pgdata:/var/lib/postgresql/data']

  redis:
    image: redis:7
    ports: ['6379:6379']

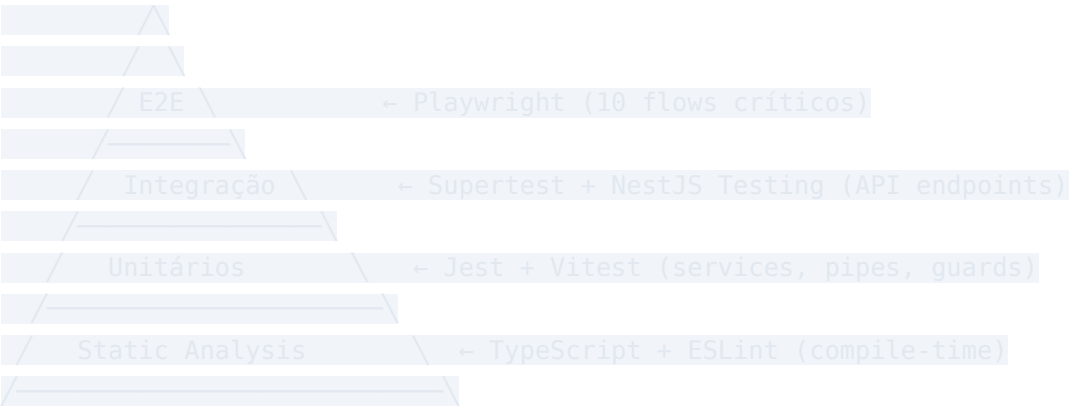
  backend:
    build:
      context: ./packages/backend
      dockerfile: ../../docker/Dockerfile.backend
    ports: ['4000:4000']
    depends_on: [postgres, redis]
    env_file: .env

  frontend:
    build:
      context: ./packages/frontend
      dockerfile: ../../docker/Dockerfile.frontend
```

```
ports: ['3000:3000']
depends_on: [backend]
env_file: .env
```

11. Estratégia de QA e Testes

Pirâmide de Testes



O que testar

Static	TypeScript + ESLint	100% files	Tipos, lint, padrões
Unitário	Jest	>70% services	Auth, Evaluation engine, validators
Integração	Supertest + NestJS	>50% endpoints	API CRUD, auth flow, permissions
E2E	Playwright	10 flows	Login, signup, CRUD entities, chat

Flows E2E Críticos

- Registo e Login** → signup → verify email → login → 2FA setup → login com 2FA
- Gestão de Perfil** → view profile → edit info → upload avatar

3. **Amizades** → send request → accept request → chat com amigo
4. **Gestão Académica (Admin)** → criar turma → criar disciplina → matricular alunos → criar exame → lançar notas
5. **Chat** → abrir conversa → enviar mensagem → typing indicator → read receipt
6. **Notificações** → receber notificação → clicar → marcar como lida
7. **Importação** → upload CSV → validar → importar → ver dados importados
8. **Admin** → criar escola → criar admin → configurar permissões
9. **GDPR** → solicitar exportação → receber email → download dados → eliminar conta
10. **Multilíngue** → mudar idioma → ver páginas em PT, EN, FR, ES

Estratégia de Testes

Unitários (M3 + M4):

```
// test/services/evaluation.service.spec.ts
describe('EvaluationService', () => {
  it('should calculate weighted average correctly')
  it('should round grades according to config')
  it('should calculate trimester classification')
  it('should reject invalid grading config')
})
```

Integração (M3 + M4):

```
// test/controllers/auth.controller.spec.ts
describe('AuthController', () => {
  it('POST /auth/signup → should create user')
  it('POST /auth/signin → should return JWT')
  it('GET /auth/profile → should return user data')
  it('POST /auth/signin → should reject invalid password')
})
```

E2E (M5):

```
// test/e2e/auth.flow.spec.ts
test('complete auth flow', async ({ page }) => {
  await page.goto('/signup')
  await page.fill('[name=email]', 'test@test.com')
  // ... complete flow
})
```

Quando testar

- **Unitários:** Durante desenvolvimento (npm run test:watch)
 - **Integração:** Antes de abrir PR (npm run test)
 - **E2E:** No CI, após deploy para staging
 - **Regressão:** Manual, antes de release (pela M5)
 - **Bug bash:** Final de cada sprint (toda a equipa)
-

12. Estrutura de Documentação

Documentos Obrigatórios

```
docs/
├── 01-ARCHITECTURE.md      # Arquitetura geral (diagramas, decisões)
├── 02-API.md               # OpenAPI/Swagger (endpoints, exemplos)
├── 03-DATABASE.md         # Schema visual, relações, índices
├── 04-SECURITY.md         # Auth flow, 2FA, CSRF, rate limiting
├── 05-DEPLOYMENT.md       # Docker, CI/CD, ambientes
├── 06-TESTING.md          # Estratégia de testes, como executar
├── 07-MODULES.md          # Módulos 42, pontos, justificação
├──
├── ADR/                   # Architecture Decision Records
│   ├── 001-use-nestjs.md
│   ├── 002-monorepo-turborepo.md
│   ├── 003-prisma-over-drizzle.md
│   └── 004-websocket-gateway.md
├──
├── SPRINTS/
│   ├── sprint-0.md
│   ├── sprint-1.md
│   └── ...
```

ADR — Architecture Decision Records

Template:

```
# ADR 001: Usar NestJS para o Backend
```


Contexto

Precisamos de um backend modular, testável e escalável para substituir as API Routes do Next.js.

Decisão

Vamos usar NestJS 11 com os seguintes módulos core:

- @nestjs/core, @nestjs/common, @nestjs/platform-express
- @nestjs/passport + @nestjs/jwt (autenticação)
- @nestjs/websockets (WebSocket gateway)
- @nestjs/swagger (documentação automática)

Consequências

Positivas: Modular, DI, decorators, testável, Swagger automático

Negativas: Curva de aprendizagem, mais boilerplate que Express

Trade-off: Maior estrutura → maior manutenibilidade a longo prazo

Status

Aceito

README.md (para avaliação)

Seguindo os requisitos do subject:

1. Primeira linha: `*This project has been created as part of the 42 curriculum by ahamuyel, [team_members...]*`
 2. Description (nome, features, goal)
 3. Instructions (prerequisites, setup, run)
 4. Resources (docs, AI usage)
 5. Team Information (roles, responsabilidades)
 6. Project Management (tools, comunicação)
 7. Technical Stack (justificação)
 8. Database Schema (visual/descrição)
 9. Features List (quem fez o quê)
 10. Modules (lista, pontos, justificação, implementação)
 11. Individual Contributions
-

13. Riscos Técnicos e Mitigação

Matriz de Riscos

1	Curva de aprendizagem NestJS	Alta	Médio	Workshop inicial de 2 dias; M1 faz pairing com M3 e M4
2	Incompatibilidade de tipos shared	Média	Alto	Contract-first; testes de tipo no CI
3	Dependências bloqueantes	Média	Alto	M1 prioriza setup de infra; cada sprint começa com módulos base
4	WebSocket escalabilidade	Baixa	Alto	Redis pub/sub desde o início; não adiar
5	Perda de dados em migração	Média	Alto	Script de migração testado em staging; backup antes de migrar
6	Escopo mal estimado	Alta	Médio	Sprints de 2 semanas com revisão de escopo; cortar features não-críticas
7	Conflitos de merge	Alta	Baixo	Branches curtas (<3 dias); monorepo com packages separados; comunicação
8	Dívida técnica da v1 replicada	Média	Médio	Code review obrigatório; ADRs documentam decisões
9	Membro da equipa ausente	Baixa	Alto	Task ownership partilhada; documentação atualizada; ninguém é único dono

1	Falha em alcançar 14	Média	Crítico	Priorizar módulos core primeiro;
0	pontos			módulos de escolha como fallback

Plano de Mitigação Detalhado

Risco 1: Curva de aprendizagem NestJS

Ação:

- Workshop de 2 dias antes do Sprint 0 (M1 apresenta)
- Criar um "guia rápido" de NestJS para a equipa
- Pair programming nas primeiras tasks de backend
- Code review focado em padrões NestJS

Risco 3: Dependências bloqueantes

Ação:

- M1 mantém um "gráfico de dependências" vivo no GitHub
- Backend e frontend podem trabalhar paralelamente usando contratos + mocks
- Se um módulo atrasar, o frontend usa mock data até o backend ficar pronto

Risco 5: Perda de dados em migração

Ação:

- Script de migração versionado (`scripts/migrate-v1-to-v2.ts`)
- Testar migração em staging com cópia dos dados de produção
- Backup completo antes de qualquer migração
- Rollback plan documentado

Risco 10: Falha em alcançar 14 pontos

Ação:

- M1+M3 fazem tracking semanal dos pontos acumulados
 - Módulos priorizados por ponto/esforço (maior retorno primeiro)
 - Meta: 16 pontos (2 de margem)
 - Se atrasar, cortar módulos menor de menor valor primeiro
-

14. Checklist de Execução

Pré-Requisitos (Antes do Dia 1)

- Todos os membros têm acesso ao GitHub
- Docker + Node.js 20 instalado em todas as máquinas
- Conta Neon (ou PostgreSQL local) configurada
- Conta Resend configurada
- Discord/Slack server criado
- GitHub Projects configurado (template: kanban)
- ADRs iniciais escritos e aceites

Sprint 0 — Setup

Dia 1-2:

- Monorepo criado (Turborepo + 3 packages)
- ESLint + Prettier + Husky configurado
- Docker Compose funcional (postgres + redis + backend + frontend)
- CI/CD pipeline básico (lint + build)
- Todos conseguem rodar `docker compose up` com sucesso

Dia 3-5:

- Prisma schema completo (30+ models)
- PrismaModule + PrismaService no NestJS
- RedisModule + RedisService no NestJS

- Common layer: AuthGuard, RolesGuard, ValidationPipe, ExceptionFilter, LoggingInterceptor
- AuthModule: signup + signin + JWT + refresh token

Dia 6-7:

- Frontend: Next.js + Tailwind + shadcn/ui configurado
- Frontend: Design System (10 componentes base: Button, Input, Table, Card, Modal, Dropdown, Badge, Avatar, Toast, Spinner)
- Frontend: Auth pages (login, register, 2FA verify)
- Documentação: ADRs + Architecture + Onboarding guide
- Workshop NestJS para a equipa (M1)

Sprint 1 — Core

Dia 8-10:

- UsersModule: CRUD users, profiles, avatar
- SchoolModule: CRUD schools, multi-tenant isolation
- Frontend: Dashboard layout (sidebar, navbar, theme switcher)
- Frontend: Profile page with avatar upload

Dia 11-14:

- FriendsModule: send/accept/block/unfriend
- NotificationModule: CRUD + WebSocket broadcast
- Frontend: Friend system UI (list, requests, search)
- Frontend: Notification center UI (dropdown, list, mark read)
- Testes: Auth + Users (unitários)
- Code review de todos os PRs

Sprint 2 — Academic

Dia 15-17:

- AcademicModule: Classes CRUD

- AcademicModule: Subjects CRUD
- AcademicModule: Courses CRUD
- AcademicModule: Enrollments

Dia 18-21:

- AcademicModule: Lessons CRUD (timetable)
- AcademicModule: Exams CRUD
- AcademicModule: Assignments + Submissions CRUD
- AcademicModule: Attendance CRUD
- Frontend: 21 CRUD list pages (table + pagination + filters)
- Frontend: Forms (basic CRUD)

Dia 22-28:

- Frontend: Detail pages (student, teacher, class, subject)
- Frontend: Calendar component (lessons + exams)
- Testes: Academic (integração)
- Code review

Sprint 3 — Evaluation + Chat

Dia 29-32:

- EvaluationModule: GradingConfig CRUD
- EvaluationModule: Result CRUD (trimester, average, classification)
- EvaluationModule: Grade calculation engine
- EvaluationModule: Certificates

Dia 33-38:

- ChatModule: WebSocket Gateway (auth, connect, disconnect, rooms)
- ChatModule: Conversation CRUD, message persistence
- ChatModule: Typing indicator, read receipts
- MessagingModule: Internal messages

- AnnouncementsModule: CRUD + priority levels

Dia 39-42:

- Frontend: Chat interface (conversations, messages, real-time)
- Frontend: Typing indicator + read receipts UI
- Frontend: Announcements display
- Testes: Evaluation + Chat
- Code review

Sprint 4 — Admin + Analytics

Dia 43-46:

- AdminModule: User management (CRUD all users)
- AdminModule: Role/permission management
- Advanced Permissions: 15 granular permissions
- Frontend: Admin panel (users, schools, config, logs)

Dia 47-52:

- AnalyticsModule: Dashboard data endpoints
- AnalyticsModule: Stats aggregation (academic, user, school)
- FilesModule: Upload (validation, preview, delete, progress)
- ImportExportModule: CSV/XLSX import + export
- GDPRModule: Data export + account deletion + confirmation emails

Dia 53-56:

- Frontend: Analytics dashboard (Recharts: line, bar, pie, area)
- Frontend: File upload UI (drag & drop, progress, preview)
- Frontend: Import modal (upload, validate, confirm)
- Frontend: GDPR settings page
- Testes: Admin + Analytics + Files
- Code review

Sprint 5 — Quality

Dia 57-60:

- i18n: PT (completo), EN (completo), FR, ES
- Public API: 5+ endpoints + rate limiting + API key auth
- Swagger/OpenAPI automático
- Privacy Policy + Terms of Service pages
- Frontend: Responsive design pass (mobile, tablet, desktop)
- Frontend: Accessibility pass (keyboard nav, ARIA labels, screen reader)

Dia 61-66:

- Testes E2E Playwright (10 flows)
- Testes unitários (cobertura >60%)
- Bug bash (toda a equipa, 1 dia)
- Performance optimization (Lighthouse >80)
- README.md completo (para avaliação)
- Script de migração v1 → v2

Dia 67-70:

- Deploy para staging
- Testes de regressão em staging
- Deploy para produção
- Smoke tests em produção
- Rollback plan verificado
- Entrega final

Anexo: Comandos Úteis

```
# Setup inicial
git clone git@github.com:ahamuyel/curl0usx.git
cd curl0usx
```



```
npm install

# Desenvolvimento
cd packages/backend && npm run start:dev
cd packages/frontend && npm run dev

# Docker
docker compose up -d
docker compose logs -f backend
docker compose down

# Prisma
npx prisma generate
npx prisma migrate dev
npx prisma studio

# Testes
npm run test          # Todos os testes
npm run test:e2e      # E2E tests
npm run test:coverage # Cobertura

# Build
npx turbo build
```

Documento mantido por: M1 (Tech Lead)

Última atualização: Maio 2026

Próxima revisão: Após Sprint 0